UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°2:
Sentencia de Manipulación de Datos

Material Teórico
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

Sentencias de Manipulación de Datos 2

Sentencia INSERT 3

Sentencia UPDATE 6

Sentencia DELETE 7

Recuperación de Datos 8

Consultas 8

Sentencia SELECT 9

La lista de selección: Cláusula SELECT 9

Ordenamiento de los resultados de una consulta 14

Condiciones de búsqueda en el WHERE 18

BIBLIOGRAFÍA 20

.

Sentencias de Manipulación de Datos

DML (Data Manipulation Language) es un conjunto de comandos utilizados para manipular los datos en una base de datos relacional. Los comandos DML se utilizan para agregar, actualizar y eliminar registros de una tabla, así como para recuperar datos de una tabla. Los comandos DML más comunes son SELECT, INSERT, UPDATE y DELETE.

Sentencia DML	Descripción
SELECT	Recupera datos de una tabla
INSERT	Agrega nuevos registros a una tabla
UPDATE	Modifica los registros existentes en una tabla
DELETE	Elimina registros de una tabla

Tabla 1: Elaboración propia.

Es importante tener en cuenta que al realizar cambios con las sentencias INSERT, UPDATE y DELETE, estos cambios no se hacen efectivos en la base de datos hasta que se ejecuta una sentencia de *Control de Transacciones*: **COMMIT**. La ejecución de COMMIT asegura que el resto de los usuarios puedan percibir las modificaciones realizadas por el usuario.

Sentencia INSERT

La sentencia INSERT es una de las sentencias más utilizadas en el lenguaje SQL para agregar nuevos datos a una tabla. Es importante tener en cuenta que la tabla debe estar previamente creada en la base de datos y que la sintaxis de la sentencia debe ser seguida cuidadosamente para evitar errores.

```
INSERT INTO Nombre_Tabla (lista_columnas)
VALUES (lista_valores)
```

En donde:

- **Nombre_tabla**: es la tabla de la base de datos donde se quieren insertar nuevos registros
- **Lista_columnas**: son columnas de la tabla, en la que se quiere insertar datos.
- **Lista_valores**: son los valores que se van a insertar en las columnas antes especificadas. ***Los valores deben coincidir con el tipo de dato de la columna correspondiente, y deben existir tantos valores como columnas se especifiquen en la lista***.

Además de los elementos ya mencionados en la sentencia INSERT, es importante tener en cuenta algunos detalles adicionales:

- Si se quiere insertar un valor de tipo cadena (VARCHAR) que contiene una comilla simple, se debe usar otra comilla simple para escapar el carácter. Por ejemplo, si se quiere insertar el valor **'Hola Mundo'**.
- Si se quiere insertar un valor de tipo fecha, se debe usar el formato DD/MM/YYYY dentro de las comillas simples. Por ejemplo, **'06/03/2023'** representa la fecha 6 de marzo de 2023.
- Si se quiere insertar un valor de tipo numérico, se debe usar el punto como separador de decimales. Por ejemplo, **3.14** representa el número pi en notación decimal.
- Si se quiere insertar un valor nulo en una columna que lo permite, se debe usar la palabra reservada **NULL** dentro de la lista de valores.

Es importante destacar que si se omite alguna columna o valor en la sentencia INSERT, se producirá un error de sintaxis y la inserción de datos no se llevará a cabo.

Por ejemplo: Dadas las tablas del diagrama y teniendo previamente registros en la tabla ***tipos_dni***, insertar un nuevo registro en la tabla ***personas***

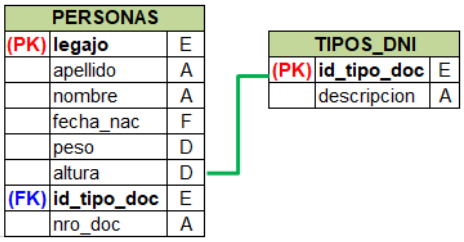

Ilustración 2: Elaboración propia.

TIPOS_DNI	
id_tipo_doc	descripcion
1	DNI
2	Pasaporte
3	LC
4	...

Ilustración 1: Elaboración propia.

La sentencia:

```
INSERT INTO PERSONAS (legajo,apellido,peso,id_tipo_doc,fecha_nac)
            VALUES (2154,'López',89.5, 1, '10/05/1997')
```

La sentencia está dirigida a la tabla "personas", la cual tiene cinco campos o columnas: "legajo", "apellido", "peso", "id_tipo_doc" y "fecha_nac".

En la sentencia INSERT, se especifican los valores que se van a insertar en cada una de estas columnas, siguiendo el orden en que aparecen en la definición de la tabla. En este caso, se está insertando un nuevo registro con los siguientes valores:

- legajo: 2154
- apellido: López
- peso: 89.5
- id_tipo_doc: 1
- fecha_nac: '10/05/1997'

Es importante destacar que la columna "legajo" es la clave primaria de la tabla "personas", lo que significa que su valor no puede estar repetido y que no puede ser nulo. Por lo tanto, siempre debe ser ingresado un valor para esta columna en una sentencia INSERT.

Asimismo, la columna "apellido" ha sido definida como "NOT NULL" en la creación de la tabla, lo que significa que su valor tampoco puede ser nulo y siempre debe ser ingresado un valor para esta columna en una sentencia INSERT.

Finalmente, se menciona que el valor del campo "id_tipo_doc" debe estar ingresado previamente en la tabla "tipos_dni", lo que sugiere que esta tabla estaría relacionada con la tabla "personas" y que la columna "id_tipo_doc" de la tabla "personas" actuaría como una clave foránea que hace referencia a la tabla "tipos_dni".

Insertar datos desde una sentencia Select

La sentencia INSERT también permite agregar múltiples filas de datos en una tabla utilizando la cláusula SELECT en lugar de especificar valores directamente en la lista de valores. El formato para realizar esta operación es:

```
INSERT INTO Nombre_Tabla1
SELECT lista_campos FROM Tabla2
```

Donde "**Nombre_Tabla1**" es la tabla de destino y "**Tabla2**" es la tabla de origen de donde se extraerán los datos a insertar. La cláusula **SELECT** se utiliza para especificar qué columnas se seleccionan de la tabla de origen y en qué orden se insertarán en la tabla de destino.

Es importante tener en cuenta que, al igual que con la sentencia INSERT normal, si los valores que se intentan insertar no cumplen con las restricciones definidas para los campos de la tabla de destino, la operación no se llevará a cabo. Además, es necesario que ambas tablas tengan columnas con tipos de datos compatibles para que se pueda realizar la operación.

Por ejemplo, si tenemos dos tablas "personas" y "personas_nuevas", donde "personas" ya contiene algunos registros y "personas_nuevas" tiene nuevos datos que deseamos agregar a la tabla "personas". Podemos usar la siguiente sintaxis:

```
INSERT INTO personas_nuevas (nombre, apellido, edad)
SELECT nombre, apellido, edad
  FROM personas
```

En este caso, estamos seleccionando los campos "nombre", "apellido" y "edad" de la tabla "personas_nuevas" y luego insertándolos en las mismas columnas correspondientes en la tabla "personas". Es importante tener en cuenta que los nombres y tipos de datos de las columnas deben coincidir entre ambas tablas para que la inserción funcione correctamente.

Si algún valor en la tabla "personas_nuevas" no cumple con las restricciones de la tabla "personas", como una clave primaria que ya existe o un valor que excede el límite de caracteres permitidos, la inserción no se llevará a cabo y se mostrará un error.

Sentencia UPDATE

La sentencia UPDATE se utiliza para modificar los datos de las columnas de una tabla en una base de datos relacional. La estructura básica de la sentencia UPDATE es la siguiente:

```
UPDATE Nombre_Tabla
   SET columna1 = Nuevo_Valor1
    [, columna2 = Nuevo_Valor2, ...]
[ WHERE condición ]
```

Donde:

- **Nombre_Tabla**: es el nombre de la tabla que queremos actualizar.
- **SET**: es la cláusula que especifica la columna y el nuevo valor que se desea establecer en la tabla, separados por una coma. Se pueden actualizar múltiples columnas en una sola sentencia.
- **columna1, columna2, ...**: son los nombres de las columnas que se quieren actualizar.
- **Nuevo_Valor1, Nuevo_Valor2, ...**: son los nuevos valores que se quieren asignar a las columnas correspondientes.
- **WHERE** (opcional): es la cláusula que permite aplicar una condición para actualizar sólo las filas que cumplen con dicha condición. **Condición**: es una expresión lógica que debe evaluarse como verdadera para que se actualice una fila. ***Si no se especifica una condición con el WHERE, la sentencia UPDATE actualiza todas las filas de la tabla.***

Por ejemplo, se está actualizando el peso y la altura de la persona con el legajo 2154 en la tabla "personas". Para ello, se utiliza la cláusula SET para establecer los nuevos valores de peso y altura a 85.5 y 1.78 respectivamente.

```
UPDATE personas
   SET peso = 85.5,
       altura = 1.78
 WHERE legajo = 2154
```

La cláusula WHERE se utiliza para especificar que solo se deben actualizar las filas que tienen un valor de legajo igual a 2154. Esto garantiza que se actualicen solo los registros necesarios y se evite la actualización de registros no deseados.

Es importante tener en cuenta que, si la actualización de una fila infringe una restricción o regla, como la configuración de valores NULL o si el nuevo valor es de un tipo de datos incompatible, la instrucción se cancela, se devuelve un error y no se actualiza ningún registro. Por lo tanto, es esencial conocer las restricciones y reglas de la base de datos al realizar actualizaciones para evitar errores y garantizar la integridad de los datos.

Si no se especifica la cláusula WHERE en el ejemplo de la sentencia UPDATE para actualizar el peso y la altura de la persona con el legajo 2154 en la tabla personas, entonces ***se actualizarán todos los registros de la tabla personas***, lo que podría provocar cambios no deseados en los datos. Esto se debe a que la sentencia UPDATE modificaría todos los registros de la tabla "personas" que contengan campos "peso" y "altura", en lugar de solo actualizar el registro correspondiente al legajo 2154. Por lo tanto, es importante siempre incluir una cláusula WHERE en la sentencia UPDATE para asegurarse de que se actualicen solo las filas deseadas y se eviten cambios no deseados en los datos.

Sentencia DELETE

Es una instrucción en SQL que permite eliminar uno o varios registros de una tabla. La sintaxis de la sentencia **DELETE** comienza con la palabra clave DELETE, seguida del nombre de la tabla donde se eliminarán los registros.

```
DELETE Nombre_tabla
[WHERE Condición ]
```

La cláusula **WHERE** es **opcional**, pero es recomendable utilizarla ya que permite especificar una condición que debe cumplirse para que se eliminen las filas correspondientes. Si no se especifica una condición con la cláusula WHERE, se borrarán todos los registros de la tabla.

En el ejemplo dado, se está eliminando el registro de la tabla "personas" que tiene un legajo igual a 3210. La sintaxis utilizada es:

```
DELETE personas
 WHERE legajo = 3210
```

Es importante tener en cuenta que, al igual que con la sentencia UPDATE, si se intenta eliminar una fila que está relacionada con otras tablas mediante claves foráneas, se puede generar un error de integridad referencial. Por lo tanto, es importante tener en cuenta las restricciones y relaciones de la base de datos al utilizar la sentencia DELETE.

En resumen, la sentencia DELETE

- permite eliminar uno o varios registros de una tabla en SQL.
- La cláusula WHERE es opcional, pero es recomendable utilizarla para asegurarse de que se eliminen solo las filas deseadas.
- Es importante tener en cuenta las restricciones y relaciones de la base de datos para evitar errores de integridad referencial.

Recuperación de Datos

Consultas

Una consulta es un conjunto de instrucciones que permite interactuar con los datos almacenados en las tablas de una base de datos. Su uso es necesario para obtener información de una o varias tablas de la base de datos, obteniendo un subconjunto de registros que cumplan con determinadas condiciones, o para ver, modificar o eliminar los datos de una tabla. Para trabajar con los datos de una base de datos, se utiliza un conjunto de comandos e instrucciones, definidos por el software del DBMS. El lenguaje SQL es el más utilizado en las bases de datos relacionales y es definido por estándares internacionales.

Realizar consultas a la base de datos es una tarea muy frecuente. Cuando las relaciones entre tablas son complejas, las consultas pueden ser difíciles de diseñar y ejecutarse en un tiempo razonable. Por lo tanto, es importante optimizar el tiempo de respuesta y diseñar consultas eficientes. El lenguaje SQL es muy flexible y permite realizar consultas de diversas formas.

Sentencia SELECT

La sentencia SELECT es una instrucción de SQL utilizada para recuperar datos de una o más tablas de una base de datos. Esta sentencia es una herramienta fundamental para expresar consultas y obtener información específica de una base de datos. Es una de las sentencias más potentes y versátiles en SQL, lo que permite realizar consultas muy complejas y adaptadas a las necesidades de los usuarios.

La sintaxis básica de la sentencia SELECT incluye una lista de columnas que se quieren recuperar y la tabla o tablas de donde se obtendrán los datos.

Es importante destacar que la sentencia ***SELECT no modifica los datos de la base de datos, sino que simplemente los recupera y los presenta en forma de tabla para su visualización o procesamiento.***

La sintaxis de la sentencia completa es:

```
  SELECT [ DISTINCT | ALL ] lista_columnas
[   INTO nueva_tabla]  --Propio de SQL Server
    FROM lista_tablas
[  WHERE condición ]
[  GROUP BY lista_columnas ]
[ HAVING condición ]
[  ORDER BY lista_columnas [ ASC | DESC ] ]
```

Se explicará en detalle cada una de las cláusulas de la sentencia SELECT en las unidades sucesivas. A continuación, se explicarán las cláusulas select, from, order by y where, de ésta última solo la condición de comparación de tal forma que se puedan realizar consultas simples.

La lista de selección: Cláusula SELECT

lista_columnas: la lista de columnas que se desean mostrar en una consulta. La lista de columnas se compone del nombre de la/s columna/s de las cuales se quiere obtener la información, separadas por comas. Esta lista recupera y muestra las columnas en el orden en el que son especificadas.

Por ejemplo, si se tienen las tablas "personas" y "tipos_dni", y se desea obtener una lista que incluya el legajo, apellido, nombre y fecha de nacimiento de la tabla "personas", se utilizará la siguiente consulta:

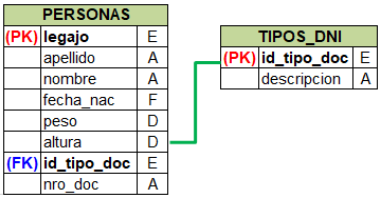

Ilustración 3: Elaboración propia.

```
SELECT legajo, apellido, nombre, fecha_nac
  FROM personas
```

La lista de columnas en la cláusula SELECT se separa por comas, y los nombres de las columnas corresponden a los nombres de los campos que pertenecen a la tabla especificada en la cláusula FROM. El resultado de esta consulta será un listado con cuatro columnas correspondientes a la información solicitada.

legajo	apellido	nombre	fecha_nac
100	Rodríguez	Juan	1/5/1996
200	Pérez	Pedro	3/8/1984
300	Pérez	Ana	12/10/2010
400	Álvarez	Juana	3/9/1977
500	Sanchez	Juan	5/8/2002

Ilustración 4: Elaboración propia.

El uso del asterisco (*) en la cláusula SELECT permite seleccionar todos los campos de una o más tablas del FROM en una consulta. Es útil en situaciones en las que se requiere obtener información completa de una tabla sin la necesidad de especificar los nombres de las columnas individualmente.

```
SELECT * FROM personas
```

El uso del asterisco (*) en la cláusula SELECT permite seleccionar todos los campos de una o más tablas del FROM en una consulta. Es útil en situaciones en las que se requiere obtener información completa de una tabla sin la necesidad de especificar los nombres de las columnas individualmente.

Además, se pueden crear columnas calculadas utilizando expresiones con operaciones matemáticas como suma, resta, multiplicación y división, así como funciones incorporadas en SQL (como COUNT, MAX y MIN que veremos más

adelante). También es posible concatenar caracteres utilizando el operador de suma **(+)** para agregar una coma y un espacio entre ellas.

Por ejemplo, si se desea generar un listado de alumnos con su nombre completo (apellido y nombre) en una sola columna, se puede utilizar el operador de suma para concatenar ambos campos en una única columna.

```
SELECT legajo, apellido + ', ' + nombre, fecha_nac
  FROM personas
```

Lo que se va a lograr, en este caso, es ver en la 2da. columna el apellido del alumno sumado a una coma y un espacio y luego el nombre; la tabla de resultado va a contener tres columnas.

legajo	(no column name)	fecha_nac
100	Rodríguez, Juan	1/5/1996
200	Pérez, Pedro	3/8/1984
300	Pérez, Ana	12/10/2010
400	Álvarez, Juana	3/9/1977
500	Sanchez, Juan	5/8/2002

Ilustración 5: Elaboración propia.

Alias: Si se presta atención al encabezado de cada columna de los ejemplos anteriores se verá que aparece en nombre del campo origen del dato o bien columna sin nombre. En SQL, los nombres de las columnas en los resultados de una consulta suelen ser el nombre del campo origen del dato o una columna sin nombre. Sin embargo, es posible cambiar estos nombres por medio de un alias.

Un alias es un nombre alternativo que se le da a una columna para que aparezca en los resultados de la consulta con un nombre diferente al original. Para agregar un alias en una consulta SQL se utiliza la palabra reservada "AS" seguida del nombre del alias, aunque en algunos casos esta palabra puede omitirse.

Por ejemplo, en la siguiente consulta se agrega un alias a la columna que concatena el apellido y el nombre para que se muestre en los resultados como "Nombre Completo":

```
SELECT legajo,
       apellido + ', ' + nombre AS 'Nombre Completo',
       fecha_nac
  FROM personas
```

legajo	Nombre Completo	fecha_nac
100	Rodríguez, Juan	1/5/1996
200	Pérez, Pedro	3/8/1984
300	Pérez, Ana	12/10/2010
400	Álvarez, Juana	3/9/1977
500	Sanchez, Juan	5/8/2002

Ilustración 6: Elaboración propia.

Es importante mencionar que, en Transact-SQL, ***actualmente no es necesaria la palabra reservada "AS"*** para agregar un alias. Por lo tanto, la consulta anterior también se puede escribir de la siguiente manera:

```
SELECT legajo,
       apellido + ', ' + nombre 'Nombre Completo',
       fecha_nac
  FROM personas
```

En este caso, la columna "Nombre Completo" tiene un alias sin comillas ya que el nombre no contiene caracteres especiales. Sin embargo, se recomienda utilizar comillas para todos los alias por razones de claridad y consistencia en el código.

DISTINCT | ALL : La cláusula "DISTINCT" es una función que se utiliza en SQL para eliminar filas duplicadas en el resultado de una consulta. Esto significa que solo se mostrarán las filas que tengan contenido único en los campos seleccionados en la consulta.

Por defecto, si no se especifica "DISTINCT" o "ALL", se utiliza la cláusula "ALL", que muestra todas las filas del resultado de la consulta, incluyendo aquellas que tengan exactamente el mismo contenido.

Para entender la diferencia entre ambas cláusulas, se pueden ejecutar dos consultas sobre la misma tabla, como se muestra a continuación:

```
SELECT peso, altura
  FROM personas
```

Esta consulta mostrará todos los registros de la tabla "personas", sin importar si existen filas duplicadas o no. Por otro lado, la siguiente consulta utiliza la cláusula "DISTINCT":

```
SELECT DISTINCT peso, altura
  FROM personas
```

Esta consulta mostrará solo las filas que tengan contenido único en los campos "peso" y "altura", es decir, eliminará las filas duplicadas en estos campos y solo mostrará una vez cada combinación única de peso y altura.

Es importante destacar que la cláusula "DISTINCT" puede hacer que la consulta sea más lenta, especialmente si se está trabajando con grandes cantidades de datos. Además, esta cláusula solo se aplica a las columnas seleccionadas en la consulta, por lo que puede ser necesario incluir más columnas si se desea eliminar filas duplicadas en base a otros criterios.

SELECT TOP

La cláusula "**SELECT TOP**" se utiliza en SQL para limitar la cantidad de filas que se devuelven en el resultado de una consulta. Al utilizar esta cláusula, se especifica el número de filas que se desean mostrar a partir del principio del resultado de la consulta.

Por ejemplo, si se quiere listar los 5 artículos más caros de una tabla de productos, se puede utilizar la cláusula "SELECT TOP" de la siguiente manera:

```
SELECT TOP 5 descripcion, pre_unitario
  FROM articulos
 ORDER BY 2 DESC
```

	descripcion	pre_unitario
1	Repuesto Gloria 400 hojas	641.00
2	Repuesto Exito 288 hojas	641.00
3	Papel p/forrar Araña * 10 u	600.00
4	Cartuchera Lata	565.50
5	Papel p/forrar Fantasía * 10 u	500.00

Ilustración 7: Elaboración propia.

En esta consulta, se está seleccionando el nombre y el precio unitario de los productos de la tabla "articulos", ordenados por precio de manera descendente (de mayor a menor). La cláusula "SELECT TOP 5" limita la cantidad de filas devueltas a 5, mostrando solo los 5 productos más caros.

En resumen, la cláusula "SELECT TOP" se utiliza en SQL para limitar la cantidad de filas devueltas por una consulta. Se especifica el número de filas deseadas después de la cláusula "SELECT TOP", y se puede combinar con la cláusula "ORDER BY" para ordenar los resultados en base a un criterio específico.

Ordenamiento de los resultados de una consulta

ORDER BY: La cláusula ORDER BY en SQL ***se utiliza para ordenar los resultados de una consulta según uno o más criterios específicos***.

La palabra reservada **ASC** (acrónimo de "ascending", que significa "ascendente" en inglés) se utiliza para ordenar los resultados en orden ascendente, es decir, de menor a mayor para columnas numéricas, en orden alfabético para columnas alfanuméricas según el código ASCII, y de la fecha más antigua a la más reciente si la columna es de tipo fecha.

ASC es el ordenamiento por defecto en la cláusula ORDER BY en SQL, lo que significa que, si no se especifica ningún ordenamiento, los resultados se ordenarán de forma ascendente según los valores de la columna indicada.

Por ejemplo, la siguiente consulta ordenaría los resultados de la tabla "personas" por apellido de forma ascendente:

```
SELECT * FROM personas ORDER BY apellido ASC;
```

La palabra reservada **DESC** (acrónimo de "descending", que significa "descendente" en inglés) se utiliza para ordenar los resultados en orden descendente, es decir, de mayor a menor para columnas numéricas, en orden alfabético inverso para columnas alfanuméricas según el código ASCII, y de la fecha más reciente a la más antigua si la columna es de tipo fecha.

Por ejemplo, la siguiente consulta ordenaría los resultados de la tabla "personas" por fecha de nacimiento de forma descendente:

```
SELECT * FROM personas ORDER BY fecha_nacimiento DESC;
```

Es importante destacar que la cláusula ORDER BY se puede utilizar con más de una columna, separando cada una con una coma. En este caso, se ordenarán los resultados primero por la primera columna indicada, y en caso de empates, por la segunda columna, y así sucesivamente.

La siguiente consulta SQL lista el legajo, apellido, nombre y fecha de nacimiento de personas, y los ordena por apellido y nombre:

```
SELECT legajo, apellido, nombre, fecha_nac
  FROM personas
 ORDER BY apellido, nombre
```

legajo	apellido	nombre	fecha_nac
400	Álvarez	Juana	3/9/1977
300	Pérez	Ana	12/10/2010
200	Pérez	Pedro	3/8/1984
100	Rodríguez	Juan	1/5/1996
500	Sanchez	Juan	5/8/2002

Ilustración 8: Elaboración propia.

En esta consulta, la cláusula ORDER BY se utiliza para ordenar los resultados de la consulta según los valores de la columna "apellido" en orden ascendente, y en caso de empate, según los valores de la columna "nombre" también en orden ascendente.

De esta forma, los resultados se mostrarán en orden alfabético primero por el apellido y luego por el nombre de cada persona. Esta cláusula es muy útil para obtener los resultados de una consulta en un orden específico y controlar la forma en que los datos son presentados.

El ordenamiento por número de columna, en la cláusula ORDER BY en SQL es una forma abreviada de ordenar los resultados de una consulta según el número de columna correspondiente. En este caso, se utiliza un número entero para hacer referencia a la posición de la columna en la lista de selección, donde el número 1 corresponde a la primera columna, el número 2 a la segunda, y así sucesivamente.

En el siguiente ejemplo, se muestra cómo ordenar el listado por la segunda columna utilizando el ordenamiento por número de columna:

```
SELECT legajo,
       apellido + ', ' + nombre 'Nombre Completo',
       fecha_nac
  FROM personas
 ORDER BY 2
```

En esta consulta, la cláusula ORDER BY se utiliza para ordenar los resultados de la consulta según los valores de la segunda columna, que en este caso es la columna "Nombre Completo". Es importante destacar que, para utilizar el ordenamiento por número de columna, es necesario que la lista de selección contenga al menos la cantidad de columnas necesarias para hacer referencia a la columna deseada.

En este ejemplo, la columna "Nombre Completo" se crea mediante una concatenación de las columnas "apellido" y "nombre". Al especificar "2" en la cláusula ORDER BY, los resultados se ordenarán según los valores de la columna "Nombre Completo".

Es importante destacar que, aunque el ordenamiento por número de columna puede ser útil en algunas situaciones, es recomendable utilizar siempre los nombres de columna en lugar de los números, ya que esto hace que el código sea más claro y fácil de entender para otros programadores.

El ordenamiento por expresión, En la cláusula ORDER BY en SQL, se puede utilizar una expresión en lugar del nombre de una columna para ordenar los resultados de una consulta. Esta expresión puede ser una función, una fórmula, o cualquier otra operación que se pueda aplicar a una columna o a varias columnas de la consulta.

En el siguiente ejemplo, se muestra cómo ordenar el listado por la segunda columna utilizando una expresión:

```
SELECT legajo,
       apellido + ', ' + nombre 'Nombre Completo',
       fecha_nac
  FROM personas
 ORDER BY apellido + ', ' + nombre
```

El ordenamiento por alias, en SQL, también es posible utilizar un alias para ordenar los resultados de una consulta en la cláusula ORDER BY. Un alias es un nombre alternativo que se le asigna a una columna o a una expresión en la lista de selección, lo cual permite referirse a ella con mayor facilidad en otras partes de la consulta, incluyendo la cláusula ORDER BY.

En el siguiente ejemplo, se muestra cómo ordenar el listado por la segunda columna utilizando un alias:

```
SELECT legajo,
       apellido + ', ' + nombre 'Nombre Completo',
       fecha_nac
  FROM personas
 ORDER BY 'Nombre Completo'
```

En esta consulta, la cláusula ORDER BY se utiliza para ordenar los resultados de la consulta según el alias "Nombre Completo", que se define como la concatenación de las columnas "apellido" y "nombre". De esta forma, los resultados se ordenarán alfabéticamente según los valores de la columna "Nombre Completo".

Es importante destacar que, cuando se utiliza un alias en la cláusula ORDER BY, el alias debe estar entre comillas simples (') o dobles ("). En este ejemplo, se utilizan comillas simples para indicar que se trata de una cadena de texto y no de una columna o una expresión.

Otro ejemplo es ordenar por apellido en forma descendente (de la Z a la A) y en nombre en forma ascendente, se puede utilizar la cláusula ORDER BY con las opciones DESC y ASC, respectivamente, para indicar el orden de cada columna.

El siguiente ejemplo muestra cómo ordenar el listado por apellido en forma descendente y por nombre en forma ascendente:

```
SELECT legajo, apellido, nombre, fecha_nac
  FROM personas
 ORDER BY apellido DESC, nombre
```

o bien,

```
SELECT legajo, apellido, nombre, fecha_nac
  FROM personas
 ORDER BY 2 DESC, 3
```

Condiciones de búsqueda en el WHERE

La cláusula WHERE en SQL se utiliza para filtrar las filas de una tabla según una o varias condiciones específicas. Si se desea seleccionar solo una parte de las filas de una tabla, las que cumplan con la condición especificada en esta cláusula, se puede utilizar la cláusula WHERE.

Al utilizar esta cláusula, SQL recorre cada una de las filas de la tabla y aplica la condición. Si la condición es verdadera para una fila determinada, esa fila se incluirá en el resultado de la consulta. De lo contrario, se excluye.

Existen varios operadores que se pueden utilizar en la cláusula WHERE para crear diferentes condiciones de búsqueda. Algunos de los más comunes son la igualdad, la comparación mayor que o menor que, entre otros.

Los ejemplos que se presentarán a continuación se basan en la base de datos librería, cuyo diagrama se muestra a continuación:

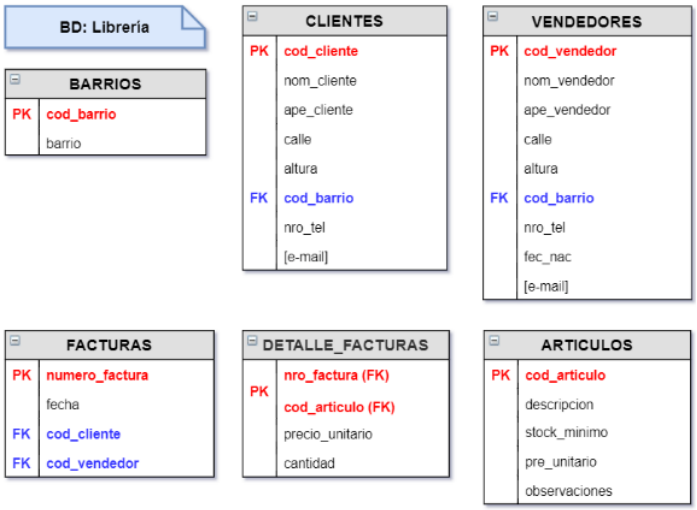

Ilustración 9: Elaboración propia.

Condiciones de comparación o Test de comparación: =, < >, !=, <, >, <=, >=

Las condiciones de comparación o test de comparación permiten a SQL calcular y comparar los valores de dos expresiones para cada fila de datos, ya sea un campo con una constante o expresiones más complejas. En este tipo de condiciones se utilizan operadores de comparación como el igual (=), distinto de (< > o !=), menor que (<), mayor que (>), menor o igual que (<=) y mayor o igual que (>=).

A continuación, se presentan algunos ejemplos de cómo se utilizan estos operadores:

- **Igual (=)**: se utiliza para comparar dos expresiones y determinar si son iguales. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'ventas' donde el valor de la columna 'monto' sea igual a 1000:

```
SELECT * FROM ventas WHERE monto = 1000
```

- **Distinto de (< > o !=)**: se utiliza para comparar dos expresiones y determinar si son distintas. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'clientes' donde el valor de la columna 'ciudad' no sea igual a 'Córdoba':

```
SELECT * FROM clientes WHERE ciudad != 'Córdoba'
```

- **Menor que (<)**: se utiliza para comparar dos expresiones y determinar si la primera es menor que la segunda. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'productos' donde el valor de la columna 'precio' sea menor que 50:

```
SELECT * FROM productos WHERE precio < 50
```

- **Mayor que (>)**: se utiliza para comparar dos expresiones y determinar si la primera es mayor que la segunda. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'empleados' donde el valor de la columna 'salario' sea mayor que 50000:

```
SELECT * FROM empleados WHERE salario > 50000
```

- **Menor o igual que (<=)**: se utiliza para comparar dos expresiones y determinar si la primera es menor o igual que la segunda. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'pedidos' donde el valor de la columna 'cantidad' sea menor o igual que 10:

```
SELECT * FROM pedidos WHERE cantidad <= 10
```

- **Mayor o igual que (>=)**: se utiliza para comparar dos expresiones y determinar si la primera es mayor o igual que la segunda. Por ejemplo, la siguiente consulta SQL devuelve todas las filas de la tabla 'proveedores' donde el valor de la columna 'valoracion' sea mayor o igual que 4:

```
SELECT * FROM proveedores WHERE valoracion >= 4
```

BIBLIOGRAFÍA

- Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK
- Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016
- Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill
- Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.